Predictive Parsing (2)

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

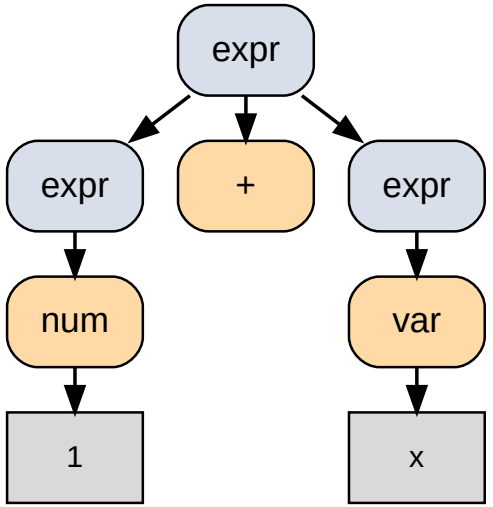
Learning outcomes

- After this lecture, you should be able to:
- derive parse trees from a BNF grammar
- define what it means for a grammar to be “ambiguous”
- use two additional rules for transforming a grammar

Parse trees

```
# mode: lhs+identifier
# terminals: num,var,+
expr ::= num | var | expr + expr
```

Parse `1 + x`



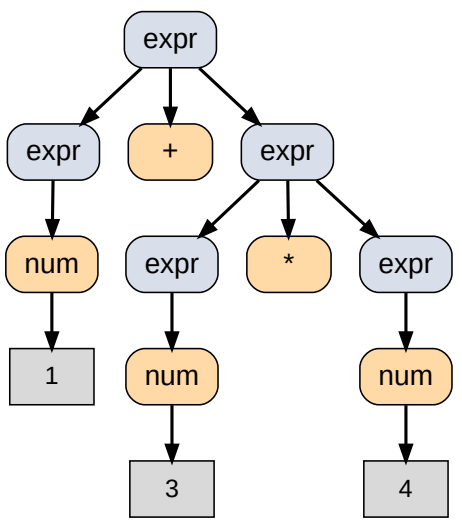
Rules:

1. root is labeled with the start symbol
2. the symbols in one of its productions become child nodes
3. continue until all leaf nodes are terminal symbols

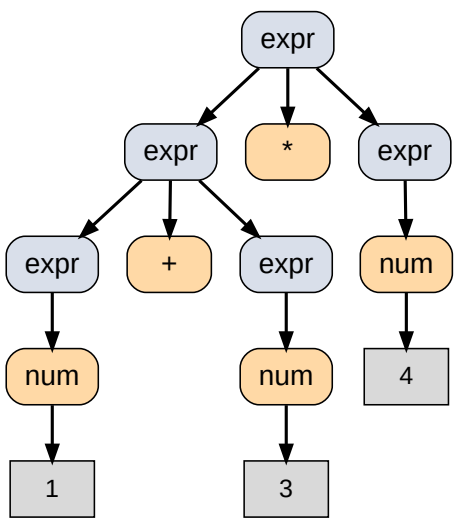
Ambiguous grammar

```
1 # mode: lhs+identifier
2 # terminals: num,+,*
3 expr ::= num | expr + expr | expr * expr
```

Create a parse tree for `1 + 3 * 4`:



`1 + (3 * 4)`



`(1 + 3) * 4`

Ambiguous grammar

- A grammar is **ambiguous** if, for some string in its language, there is more than one parse tree.
- Different parse trees usually suggest different meanings.
- So: we usually want unambiguous grammars.

Review: predictive parsing

```
# mode: lhs+identifier
# terminals: print,id,num
stmt ::= print ( expr ) | id = expr
expr ::= id | num
```

- We need to be able to pick a production of a non-terminal based on the next token.

Transforming a BNF grammar

- In the last lecture we learned how to deal with:
- left recursion
- empty productions

Two more ways to transform BNF:

- left factoring
- expanding a non-terminal

It's important to get comfortable with modifying grammars.

Left factoring

```
# mode: lhs+identifier
# terminals: a,b
A ::= a B | a C | b D
```

How about this replacement? Is it the same?

```
# mode: lhs+identifier
# terminals: a,b
A ::= a E | b D
E ::= B | C
```

Exercise

Suppose an expression can be a variable or a function call.

- Examples: `tot` (variable), `max(y)` (function call)

BNF:

```
# mode: lhs+identifier
# terminals: var,(,)
expr ::= var ( expr ) | var
```

Exercise: Apply left factoring.

Solution

```
# mode: lhs+identifier
# terminals: var,(,)
expr ::= var expr1
expr1 ::= ( expr ) | ""
```

Expanding a non-terminal

```
# mode: lhs+identifier
# terminals: a,c,d
A ::= a B | E
E ::= c C | d D
```

Any problems with a predictive parser here?

Solution

```
# mode: lhs+identifier
# terminals: a,c,d
A ::= a B | c C | d D
```

Rewriting BNFs

- For predictive parsers, we want that, for every non-terminal in the grammar:
- there's only one production for it, or
- each production starts with a terminal token (maybe `" "` on its own) and each of those tokens are different

Exercise

```
# mode: lhs+identifier
# terminals: ID,=,(,),;
prog ::= stmt | stmt ; prog
stmt ::= ID = expr | ID ( expr )
```

- There are two problems. What are they?
- Apply left factoring.

Solution

```
# mode: lhs+identifier
# terminals: ID,=,(,),;
prog ::= stmt prog1
prog1 ::= ; prog | ""
stmt ::= ID stmt1
stmt1 ::= = expr | ( expr )
```

Exercise

Expand a non-terminal (inline `stmt` into `prog`):

```
# mode: lhs+identifier
# terminals: ID,=,(,),;
prog ::= stmt prog1
prog1 ::= ; prog | ""
stmt ::= ID stmt1
stmt1 ::= = expr | ( expr )
```

Solution

```
# mode: lhs+identifier
# terminals: ID,=,(,),;
prog ::= ID stmt1 prog1
prog1 ::= ; prog | ""
stmt1 ::= = expr | ( expr )
```

Note: there is no longer a `stmt` non-terminal.

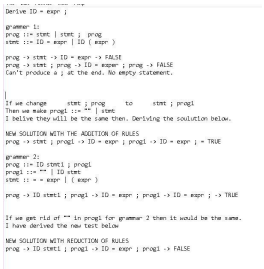
Fixed grammar

When expanding non-terminals, verify you didn't change the language.

Example check:

- Original `prog ::= stmt | stmt ; prog` does **not** derive `ID = expr ;` (no empty statement at the end).
- So the transformed grammar should not allow a trailing `;` either.

```
# mode: lhs+identifier
# terminals: ID,=,(,),;
prog ::= ID stmt1 prog1
prog1 ::= ; prog | ""
stmt1 ::= = expr | ( expr )
```



Fixed Grammar by an old TA

Summary

- A parse tree shows how a string can be derived from a BNF grammar.
- To use predictive parsing, our BNF syntax must be in a certain form. Tools to get BNF in that form:
- eliminate left recursion
- left factoring
- expanding non-terminals